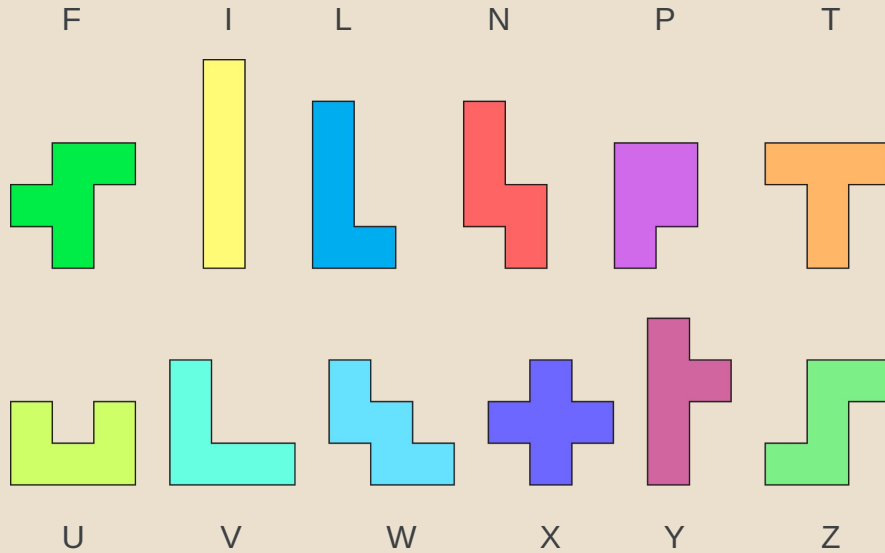
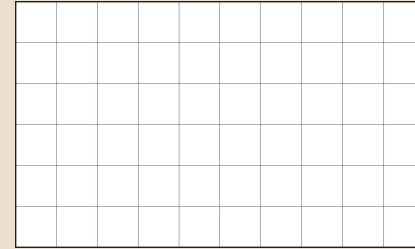

(Completely) Solving Polyomino Puzzles



5 squares per piece
12 pieces
60 squares total

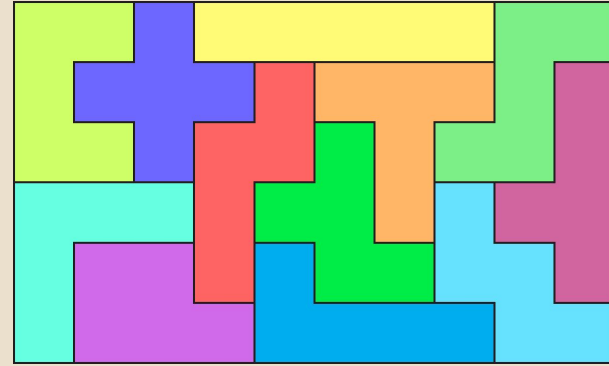
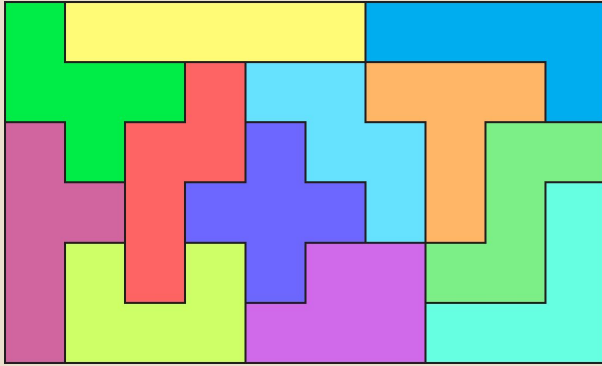
Fit them into, say, a 6 x 10 rectangle



2

Example: Pentominoes

There are multiple solutions,



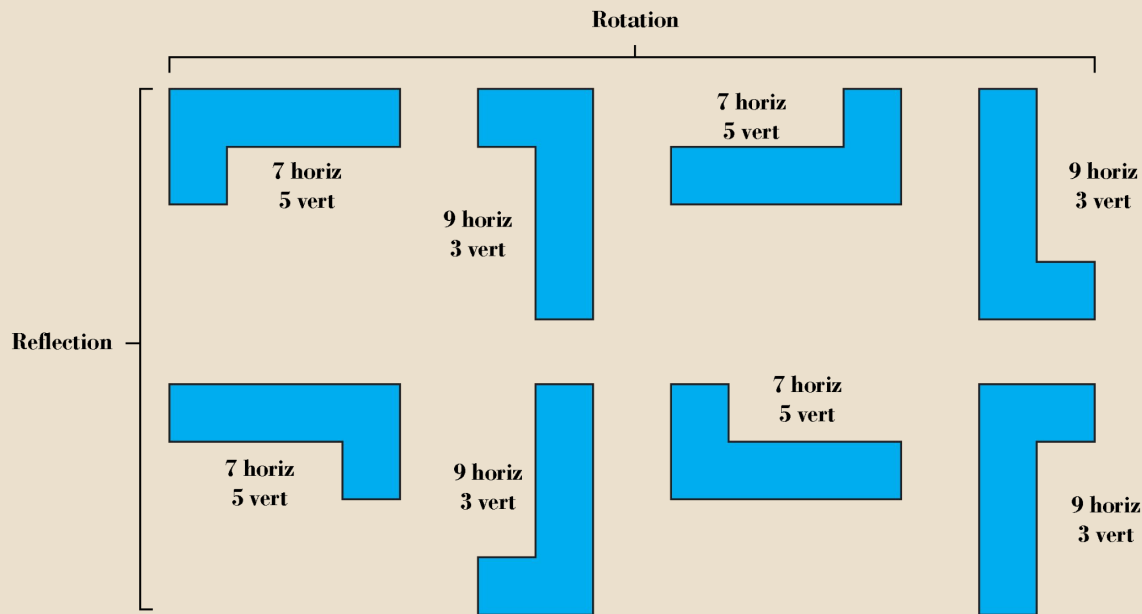
3

but how do we find them all?

A piece typically has different orientations in which it can be placed.

The “L” piece, for example, has 8. Of these, 4 span four squares horizontally and two vertically and 4 that span two squares horizontally and four vertically.

Its first orientation has 35 positions: 7 by sliding horizontally, 5 vertically.

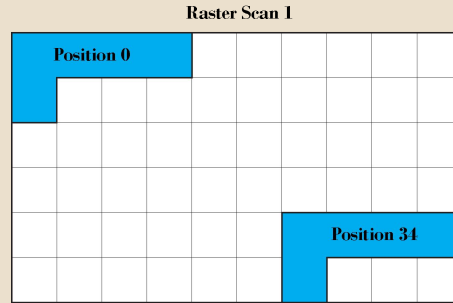


4

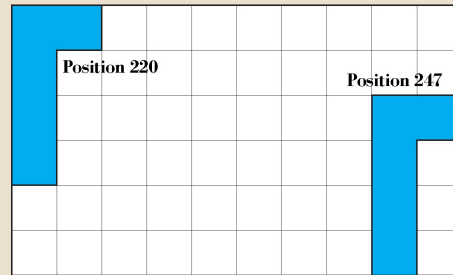
Enumerate Piece Positions

We number the positions by “raster scanning” through each of the orientations of each piece.

For example, the “L” piece has 8 orientations, and so needs 8 scans. In the end it has 248 possible positions on the 6 x 10 field.



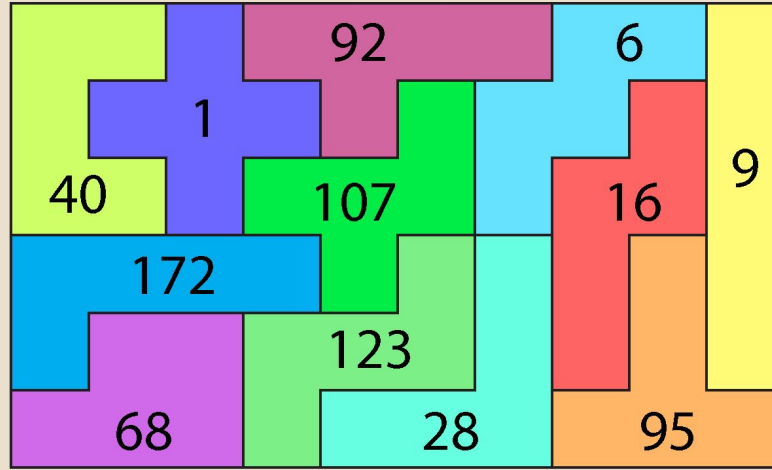
...



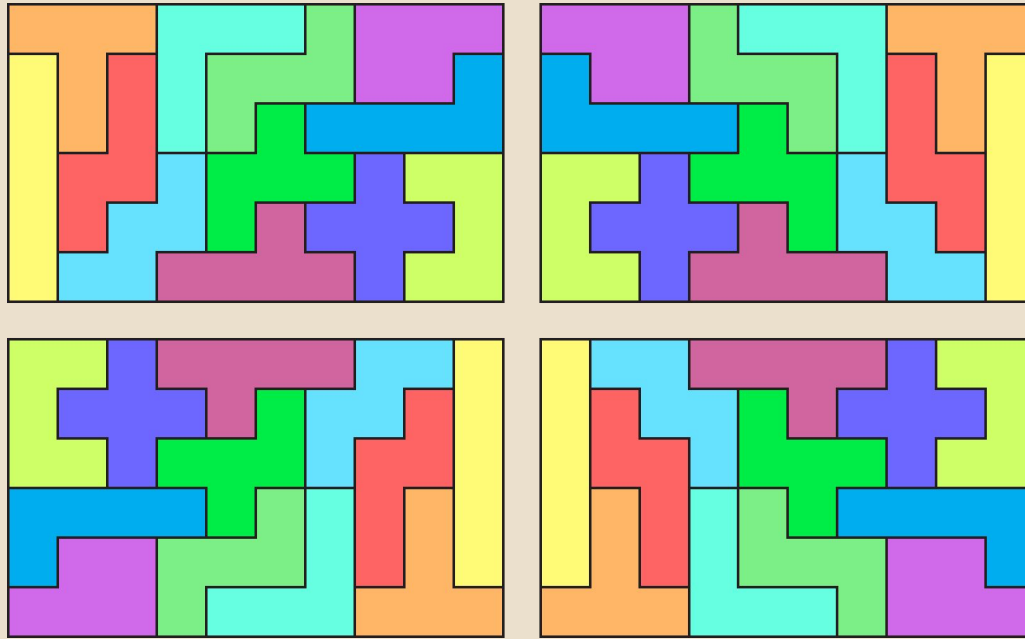
Raster Scan 8

A similar analysis gives

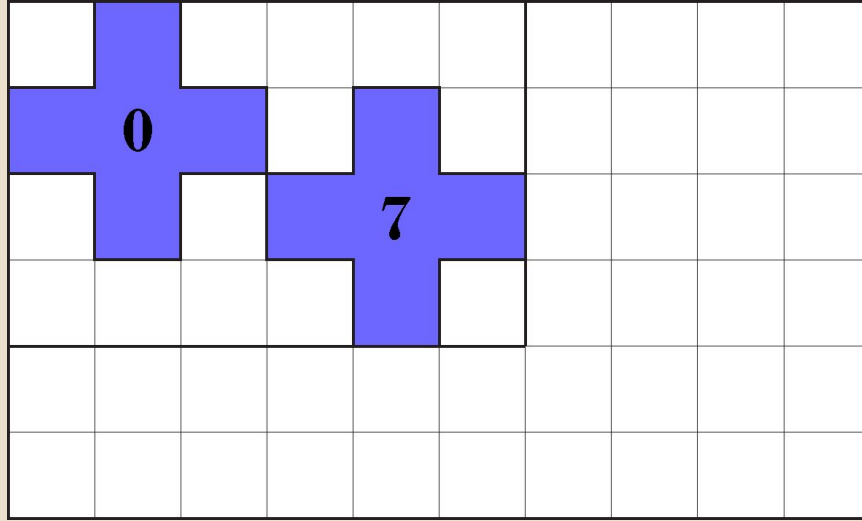
F = 256
I = 68
L = 248
N = 248
P = 304
T = 128
U = 152
V = 128
W = 128
X = 32
Y = 248
Z = 128



We can give every solution a unique “Serial Number”



But what about rotations and reflections?



8

Limit the X-piece's range

So, to find all the solutions just count from

000.000.000.000.000.000.000.000.000.000.000

to

255.067.247.247.307.127.151.127.127.007.247.127

while checking each to see if it's a solution.

But $256 \times 68 \times 248 \times 248 \times 308 \times 128 \times 152 \times 128 \times 128 \times 8 \times 248 \times 128 \approx 3.3 \times 10^{23}$ is a lot

So how do we trim the number of positions we actually have to look at without missing any solutions?

Depth-first search starting with the highest-order digit of the serial number

9

Just run through the possibilities

Classical “backtracking”

root() Return the first position of the high-order digit

reject(x) Return false if x is not worth pursuing.

accept(x) Return true if x is a solution

first(x) Return the first position of the next-order digit

next(x) Return the next position of the current-order digit.

output(x) Output the solution c

backtrack(root())

```
procedure backtrack(c)
  if reject(c) then return
  if accept(c) then output(c)
  s ← first(c)
  while s ≠ NULL do
    backtrack(s)
    s ← next(s)
```

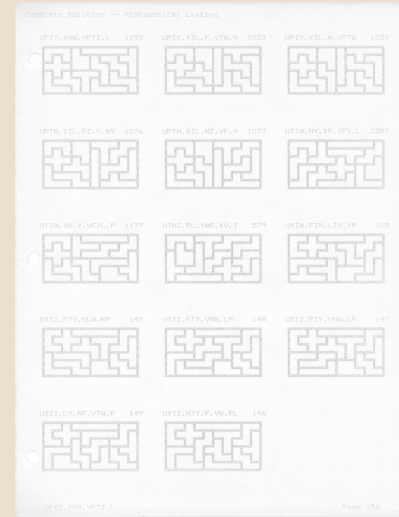
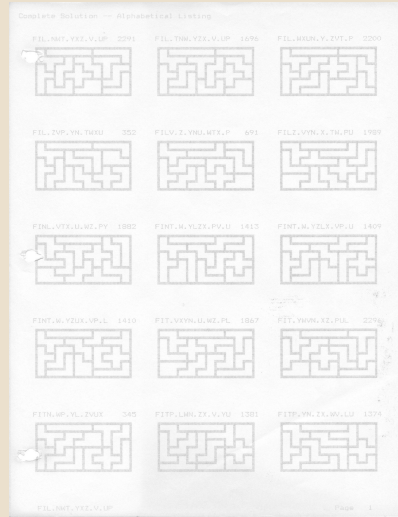
10

Key: Optimise reject(x)

Classical “backtracking”

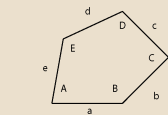
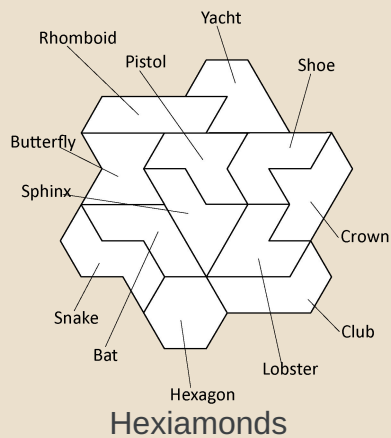
1. Place each piece in turn, starting with the piece in the serial number's high-order position.
 - a. If the piece at hand can be placed, recurse with the next lower-order piece.
 - b. If the piece at hand can't be placed, return to moving the next higher-order piece.
2. Choose the order of the pieces in the serial number so rule 1b happens early and often.
I chose X, I, W, V, T, Z, U, Y, N, L, F, P.
3. Define “can be placed” to include that the area of every “hole” is a multiple of 5 squares.

In a few weeks of 8086 time

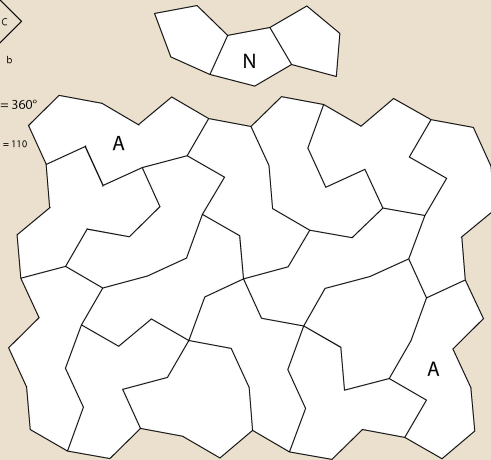


12

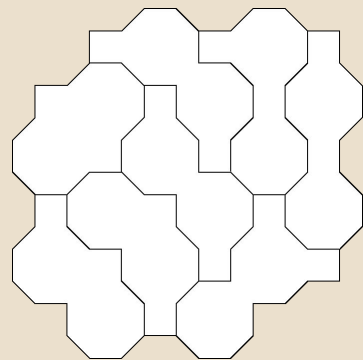
found 2,339 solutions



Type 8 : $2B + C = D + 2E = 360^\circ$
 $b = c = d = e$
 $B = 135, E = 125; C = 90, D = 110$



TriPentag



Two-Flasks

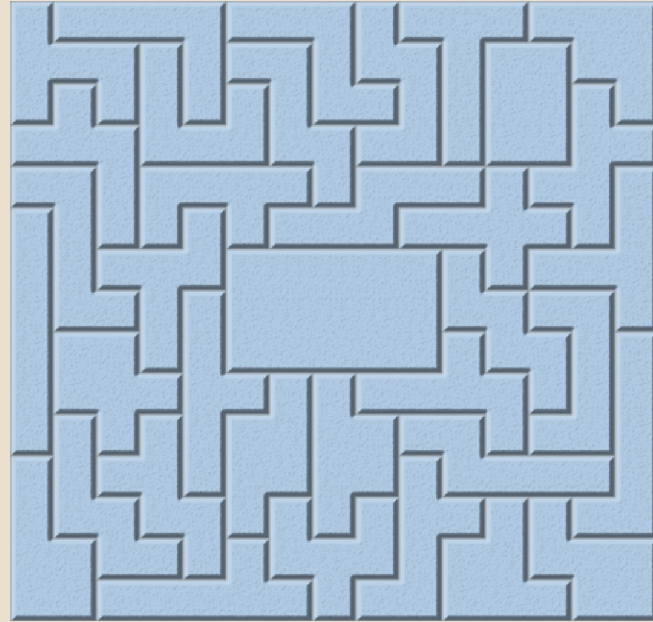
There are 35 distinct pieces. So,
 $12 * 6 = 210$ squares

They can't be placed in a
rectangle. (Next slide.)

They can be placed in other
shapes. For example, a 15 x 15
square with a 3 by 5 hole in the
middle.

It's much harder to find even
one solution by hand than the
10 by 6 pentominoes puzzle.

Still, how many are there?

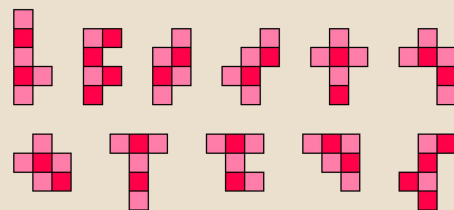
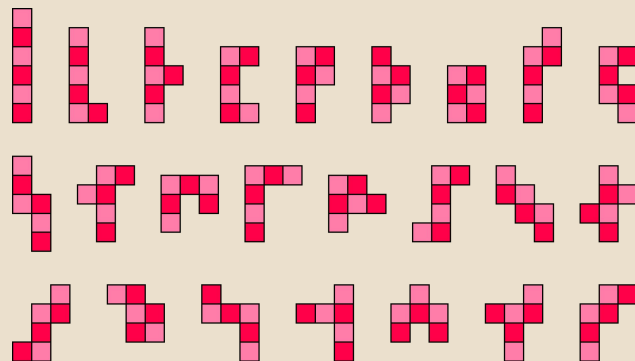


A rectangular checkerboard of 210 squares has 105 squares of each color, an odd number.

The top group of 24 hexominoes each have 3 squares of a given color. So they cover 72, an even number, of each color when all are placed on a checkerboard.

The bottom group each cover two squares of one color and 4 of the other, both even numbers.

So, taken together they cover an even number of squares of each color on a checkerboard.

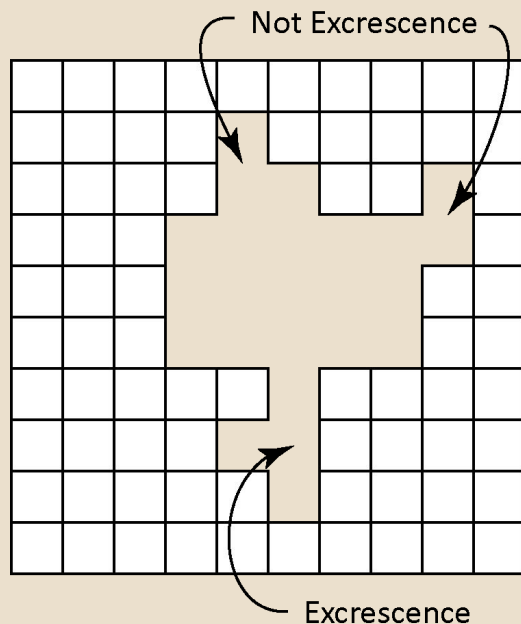


I built a solver for hexominoes. It uses the backtrack algorithm, with one addition:

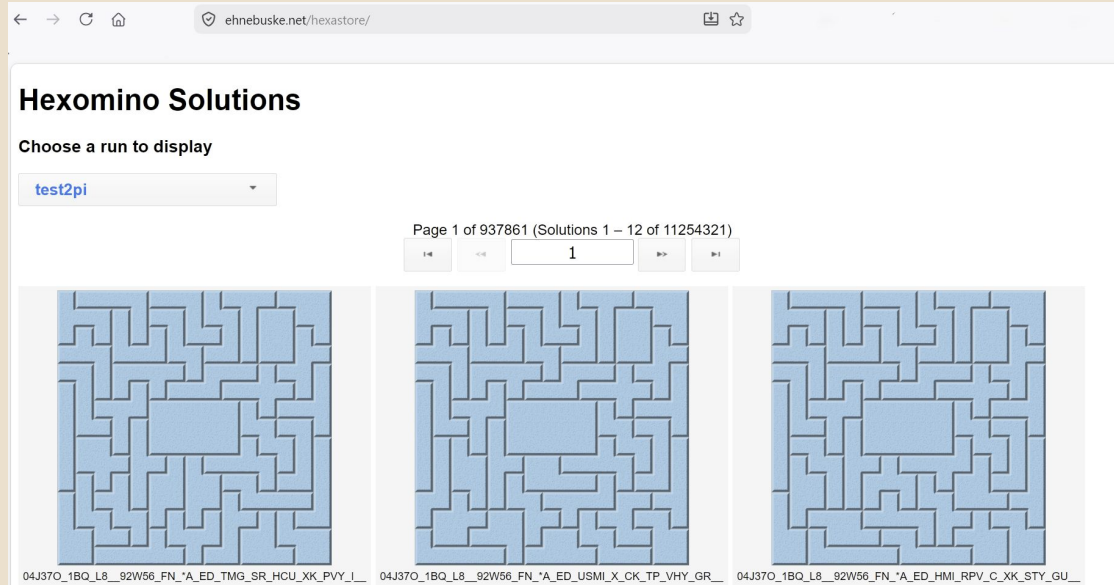
4. If placing a piece leaves one or more excrescences, make filling them — in all the ways possible — part of placing the piece.

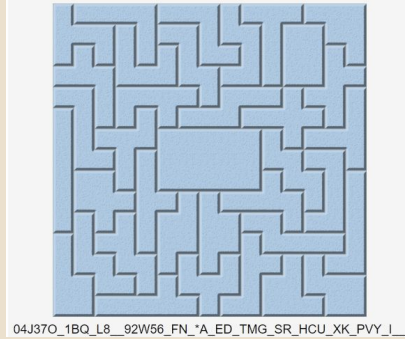
An **excrescence** is a region of three to six empty squares connected to a region of six or more empty squares by a passage of exactly one square.

Additionally, to support the solver, I built a suite of web services to store and display the solutions.

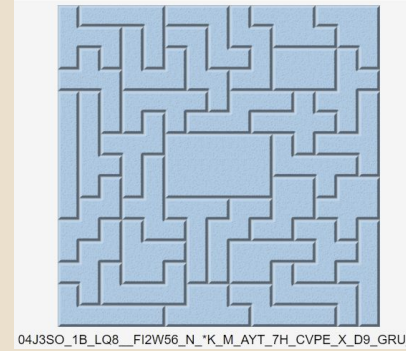


I installed the solver on a RaspberryPi 3 and ran it for 5 years. It found more than 11 million solutions.





Solution 1



Solution 11,254,321

After five years and 11 million solutions, the four high-order pieces are still in their initial positions.

Assuming the solutions are more or less evenly distributed in the algorithm's search space, that means a complete search would run $\sim 10^{10}$ years and produce $\sim 10^{17}$ solutions.

Any ideas for a

- Big optimization I missed?
- Fundamentally different algorithm?
- Radically different problem representation?

19

Help!